

WebRTC

dr inž. Robert Krupiński

html

```
1  <!DOCTYPE html>
2  <html>
3
4  <head>
5      <meta charset="utf-8">
6      <meta content="IE=edge" http-equiv="X-UA-Compatible">
7      <meta content="width=device-width, initial-scale=1" name="viewport">
8      <title>WebRTC</title>
9
10     <script src="WebRTC.js"></script>
11     <style>
12         video {
13             border: 1px solid black;
14             max-width: 120px;
15             max-height: 80px;
16         }
17     </style>
18 </head>
19
20 <body>
21     Computer A
22     <br />
23     <video id="idVideoLocalA" muted autoplay></video>
24     <video id="idVideoRemoteB" muted autoplay></video>
25     <br />
26     <input type="button" id="idButtonStart" value="Start" />
27     <input type="button" id="idButtonCall" value="Call" disabled="disabled" />
28     <input type="button" id="idButtonHangUp" value="Hang Up" disabled="disabled" />
29     <hr />
30
31     Computer B
32     <br />
33     <video id="idVideoLocalB" muted autoplay></video>
34     <video id="idVideoRemoteA" muted autoplay></video>
35
36 </body>
37
38 </html>
```

WebRTC.js

```
1 document.addEventListener("DOMContentLoaded", onReady)
2
3 function onReady() {
4
5     const
6         aVideoLocalA = document.getElementById("idVideoLocalA"),
7         aVideoRemoteB = document.getElementById("idVideoRemoteB"),
8         aVideoLocalB = document.getElementById("idVideoLocalB"),
9         aVideoRemoteA = document.getElementById("idVideoRemoteA"),
10        aButtonStart = document.getElementById("idButtonStart"),
11        aButtonCall = document.getElementById("idButtonCall"),
12        aButtonHangUp = document.getElementById("idButtonHangUp");
13
14    aButtonStart.onclick = () => {
15        start()
16    }
17    aButtonCall.onclick = () => {
18        callToUser()
19    }
20    aButtonHangUp.onclick = () => {
21        hangUpCall()
22    }
```

WebRTC.js

```
25     let aMediaStreamLocalA,  
26         aRTCPeerConnection_ComputerA,  
27         aRTCPeerConnection_ComputerB;  
28  
29  
30     function start() {  
31         aButtonStart.disabled = true  
32  
33         navigator.mediaDevices.getUserMedia({ audio: true, video: true })  
34             .then(aMediaStream => {  
35                 aVideoLocalA.srcObject = aMediaStream  
36                 aMediaStreamLocalA = aMediaStream  
37  
38                 aButtonCall.disabled = false  
39             })  
40             .catch(err => {  
41                 console.error("start:err=", err)  
42             })  
43     }
```

WebRTC.js

```
45 function callToUser() {
46     if (!aMediaStreamLocalA) {
47         console.error("callToUser:!aMediaStreamLocalA")
48         return
49     }
50     aButtonCall.disabled = true
51     aButtonHangUp.disabled = false
52
53     const
54         avVideoTracks = aMediaStreamLocalA.getVideoTracks(),
55         avAudioTracks = aMediaStreamLocalA.getAudioTracks();
56
57     if (0 < avVideoTracks.length) {
58         console.log(`Using video device: ${avVideoTracks[0].label}`)
59     }
60     if (0 < avAudioTracks.length) {
61         console.log(`Using audio device: ${avAudioTracks[0].label}`)
62     }
63
64     createPeerConnection_ComputerA()
65
66     console.log("ComputerA add Transceivers for RTCPeerConnection")
67     const avTracks = aMediaStreamLocalA.getTracks();
68     avTracks.forEach(aTrack => {
69         aRTCPeerConnection_ComputerA.addTransceiver(aTrack, { streams: [aMediaStreamLocalA] })
70     })
71 }
```

WebRTC.js

```
73 function createPeerConnection_ComputerA() {
74     console.log("ComputerA create RTCPeerConnection")
75     aRTCPeerConnection_ComputerA = new RTCPeerConnection()
76     aRTCPeerConnection_ComputerA.onnegotiationneeded = cbNegotiationNeeded_ComputerA
77     aRTCPeerConnection_ComputerA.onicecandidate = cbICECandidate_ComputerA
78     aRTCPeerConnection_ComputerA.ontrack = cbTrack_ComputerA
79     aRTCPeerConnection_ComputerA.oniceconnectionstatechange = cbICEConnectionStateChange_ComputerA
80     aRTCPeerConnection_ComputerA.onicegatheringstatechange = cbICEGatheringStateChange_ComputerA
81     aRTCPeerConnection_ComputerA.onsignalingstatechange = cbSignalingStateChange_ComputerA
82     aRTCPeerConnection_ComputerA.onconnectionstatechange = cbConnectionStateChange_ComputerA
83 }
84
85 function createPeerConnection_ComputerB() {
86     console.log("ComputerB create RTCPeerConnection")
87     aRTCPeerConnection_ComputerB = new RTCPeerConnection()
88     //aRTCPeerConnection_ComputerB.onnegotiationneeded = cbNegotiationNeeded_ComputerB
89     aRTCPeerConnection_ComputerB.onicecandidate = cbICECandidate_ComputerB
90     aRTCPeerConnection_ComputerB.ontrack = cbTrack_ComputerB
91     aRTCPeerConnection_ComputerB.oniceconnectionstatechange = cbICEConnectionStateChange_ComputerB
92     aRTCPeerConnection_ComputerB.onicegatheringstatechange = cbICEGatheringStateChange_ComputerB
93     aRTCPeerConnection_ComputerB.onsignalingstatechange = cbSignalingStateChange_ComputerB
94     aRTCPeerConnection_ComputerB.onconnectionstatechange = cbConnectionStateChange_ComputerB
95 }
96
97 function cbICECandidate_ComputerA(event) {
98     sendICECandidateToComputerB(event.candidate)
99 }
100 function cbICECandidate_ComputerB(event) {
101     sendICECandidateToComputerA(event.candidate)
102 }
```

WebRTC.js

```
104 function cbNegotiationNeeded_ComputerA() {
105     console.log("ComputerA createOffer")
106     aRTCPeerConnection_ComputerA.createOffer()
107         .then(offer => aRTCPeerConnection_ComputerA.setLocalDescription(offer))
108         .then(() => {
109             sendOfferToComputerB(aRTCPeerConnection_ComputerA.localDescription)
110         })
111         .catch(err => {
112             console.error("cbNegotiationNeeded_ComputerA:err=", err)
113         })
114 }
115
116 function cbTrack_ComputerA(event) {
117     console.log("ComputerA received remote stream from ComputerB")
118     const aMediaStream = event.streams[0];
119     if (aVideoRemoteB.srcObject !== aMediaStream) {
120         aVideoRemoteB.srcObject = aMediaStream
121     }
122 }
123 function cbTrack_ComputerB(event) {
124     console.log("ComputerB received remote stream from ComputerA")
125     const aMediaStream = event.streams[0];
126     if (aVideoRemoteA.srcObject !== aMediaStream) {
127         aVideoRemoteA.srcObject = aMediaStream
128     }
129 }
```

WebRTC.js

```
131 function cbICEConnectionStateChange_ComputerA() {
132 |   console.log("ComputerA ICEConnectionState=" + aRTCPeerConnection_ComputerA.iceConnectionState)
133 | }
134 function cbICEConnectionStateChange_ComputerB() {
135 |   console.log("ComputerB ICEConnectionState=" + aRTCPeerConnection_ComputerB.iceConnectionState)
136 | }
137
138 function cbICEGatheringStateChange_ComputerA() {
139 |   console.log("ComputerA ICEGatheringState=" + aRTCPeerConnection_ComputerA.iceGatheringState)
140 | }
141 function cbICEGatheringStateChange_ComputerB() {
142 |   console.log("ComputerB ICEGatheringState=" + aRTCPeerConnection_ComputerB.iceGatheringState)
143 | }
144
145 function cbSignalingStateChange_ComputerA() {
146 |   console.log("ComputerA SignalingState=" + aRTCPeerConnection_ComputerA.signalingState)
147 | }
148 function cbSignalingStateChange_ComputerB() {
149 |   console.log("ComputerB SignalingState=" + aRTCPeerConnection_ComputerB.signalingState)
150 | }
151
152 function cbConnectionStateChange_ComputerA() {
153 |   console.log("ComputerA ConnectionState=" + aRTCPeerConnection_ComputerA.connectionState)
154 | }
155 function cbConnectionStateChange_ComputerB() {
156 |   console.log("ComputerB ConnectionState=" + aRTCPeerConnection_ComputerB.connectionState)
157 | }
```


WebRTC.js

```
159 function sendICECandidateToComputerB(aRTCIceCandidate) {
160     console.log("ComputerA send ICECandidate to ComputerB")
161     //console.log(aRTCIceCandidate)
162     if (!aRTCPeerConnection_ComputerB) {
163         console.error("sendICECandidateToComputerB:!aRTCPeerConnection_ComputerB")
164         return
165     }
166     aRTCPeerConnection_ComputerB.addIceCandidate(aRTCIceCandidate)
167         .then(() => {
168             console.log("ComputerB addIceCandidate")
169         })
170         .catch(err => {
171             console.error("sendICECandidateToComputerB:err=", err)
172         })
173 }
174 function sendICECandidateToComputerA(aRTCIceCandidate) {
175     console.log("ComputerB send ICECandidate to ComputerA")
176     if (!aRTCPeerConnection_ComputerA) {
177         console.error("sendICECandidateToComputerA:!aRTCPeerConnection_ComputerB")
178         return
179     }
180     aRTCPeerConnection_ComputerA.addIceCandidate(aRTCIceCandidate)
181         .then(() => {
182             console.log("ComputerA addIceCandidate")
183         })
184         .catch(err => {
185             console.error("sendICECandidateToComputerA:err=", err)
186         })
187 }
```

WebRTC.js

```
189 function sendOfferToComputerB(aRTCSessionDescription) {
190     console.log("ComputerA sendOffer to ComputerB")
191     //console.log(aRTCSessionDescription)
192
193     if (!aRTCPeerConnection_ComputerB) {
194         createPeerConnection_ComputerB()
195     }
196
197     aRTCPeerConnection_ComputerB.setRemoteDescription(aRTCSessionDescription)
198         .then(() => {
199             console.log("ComputerB createAnswer")
200             return aRTCPeerConnection_ComputerB.createAnswer()
201         })
202         .then(answer => aRTCPeerConnection_ComputerB.setLocalDescription(answer))
203         .then(() => {
204             sendAnswerToComputerA(aRTCPeerConnection_ComputerB.localDescription)
205         })
206         .catch(err => {
207             console.error("sendOfferToComputerB:err=", err)
208         })
209 }
```

WebRTC.js

```
211 function sendAnswerToComputerA(aRTCSessionDescription) {
212     console.log("ComputerB sendAnswer to ComputerA")
213     if (!aRTCPeerConnection_ComputerA) {
214         console.error("sendAnswerToComputerA:!aRTCPeerConnection_ComputerA")
215         return
216     }
217     aRTCPeerConnection_ComputerA.setRemoteDescription(aRTCSessionDescription)
218 }
219
220 function hangUpCall() {
221     if (aRTCPeerConnection_ComputerA) {
222         aRTCPeerConnection_ComputerA.close()
223         aRTCPeerConnection_ComputerA = null
224     }
225     if (aRTCPeerConnection_ComputerB) {
226         aRTCPeerConnection_ComputerB.close()
227         aRTCPeerConnection_ComputerB = null
228     }
229     aButtonCall.disabled = false
230     aButtonHangUp.disabled = true
231 }
232 }
```

WebRTC

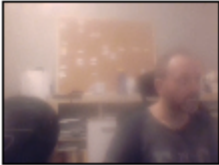
Plik D:/ProjectsDoc/wyklady/AplikacjeInternetowe/WebRTC/WebRTC.html

Computer A



Start Call Hang Up

Computer B



Elements Console Recorder Sources Network Performance insights

top Filter

```
Using video device: USB2.0 HD UVC WebCam (13d3:56a2)
Using audio device: Domyślny - Microphone (Realtek High Definition Audio)
ComputerA create RTCPeerConnection
ComputerA add Transceivers for RTCPeerConnection
ComputerA createOffer
ComputerA SignalingState=have-local-offer
ComputerA sendOffer to ComputerB
ComputerB create RTCPeerConnection
ComputerA ICEGatheringState=gathering
ComputerB SignalingState=have-remote-offer
2 ComputerB received remote stream from ComputerA
ComputerB createAnswer
2 ComputerA send ICECandidate to ComputerB
ComputerB addIceCandidate
ComputerB SignalingState=stable
ComputerB sendAnswer to ComputerA
ComputerB addIceCandidate
```

File location

- www.rmaes.com/students/2023_2024/WebRTC.pdf